

Q.No:10

Problem Description

Mathison has uncovered an ancient piece of paper on which are written N integers. This set of numbers will be referred to as A . Mathison learned in History class that a group of integers is unique if and only if their total is divisible by a mythological constant M . Mathison tries to find out how many **distinct** triplets of numbers, from the piece of paper, have their sum divisible by M . Unfortunately, this problem is quite hard to crack and he needs your help.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 10^4$
- $0 \leq A[i] \leq 2 \times 10^9$

Input

The first line of the input file contains two space-separated integers, N and M , representing the number of integers and the mythical constant. The next line contains N space-separated integers, where the i^{th} integer represents $A[i]$.

Output

The output file should contain only one integer, the answer to Mathison's question.

Explanation for Test case 1

There are 26 special trios: {0, 1, 2}; {0, 1, 8}; {0, 2, 5}; {0, 2, 6}; {0, 2, 9}; {0, 3, 7}; {0, 5, 8}; {0, 6, 8}; {0, 8, 9}; {1, 2, 7}; {1, 3, 4}; {1, 5, 6}; {1, 5, 9}; {1, 6, 9}; {1, 7, 8}; {2, 4, 8}; {2, 5, 7}; {2, 6, 7}; {2, 7, 9}; {3, 4, 5}; {3, 4, 6}; {3, 4, 9}; {5, 6, 9}; {5, 7, 8}; {6, 7, 8}; {7, 8, 9};

Note: Here we only show the **positions** (0-indexed) in each trio.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
10 5
1 10 4 3 2 5 0 1 9 5
```

EXPECTED OUTPUT

```
26
```

Test Case 2

INPUT (STDIN)

```
10 5
11 10 14 31 21 15 10 11 9 51
```

EXPECTED OUTPUT

```
31
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(i<k)
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int N, M;

    scanf("%d %d", &N, &M);

    long long *freq = (long long *)calloc(M, sizeof(long long));

    for (int i = 0; i < N; i++) {
        long long x;
        scanf("%lld", &x);
        freq[x % M]++;
    }

    long long ans = 0;

    for (int i = 0; i < M; i++) {
        for (int j = i; j < M; j++) {

            int k = (M - (i + j) % M) % M;
```

```
if (k < j || k >= M)
continue;
if (i == j && j == k) {
if (freq[i] >= 3) {
ans += freq[i] * (freq[i] - 1) *
(freq[i] - 2) / 6;
}
}

else if (i == j) {
if (freq[i] >= 2) {
ans += freq[i] * (freq[i] - 1) / 2 *
freq[k];
}
}
else if (j == k) {
if (freq[j] >= 2) {
ans += freq[i] * freq[j] *
(freq[j] - 1) / 2;
}
}
else {
ans += freq[i] * freq[j] * freq[k];
}
}
}

printf("Output:%lld\n", ans);

free(freq);

return 0;
}
```

Output:

Test_case:1

```
10 5
1 10 4 3 2 5 0 1 9 5
Output:26
```

Test_case:2

```
10 5
11 10 14 31 21 15 10 11 9 51
Output:31
```